# Lab Handout 1-F

## Cooperative Target Firmware: Bidirectional FSM with PAUSE Listener

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

## 1 Objective

Promote the Lab Handout 1-D ping-pong skeleton into production-quality cooperative-target firmware. Both Target A and Target B run the same Finite State Machine (FSM) skeleton (`INIT`→`CONFIG`→`IDLE`→`BUILD`→`TX`→`WAIT_ACK`→`LOG`→`IDLE`) with role-specific branches (Target A initiates DATA on a 1 s schedule and waits for ACKs; Target B is normally in `RX_LISTEN` and traverses `BUILD_ACK`→`TX_ACK` on every successful DATA reception). Both firmwares add a shared `PAUSED` branch entered from any state on receipt of a PAUSE packet from the interceptor (Threat) addressed to the target's own Beacon ID; while paused, the firmware skips its scheduled DATA transmissions and prints `"I'm Jammed"` on USART2 once per pause-entry. Validate two-target ping-pong over a 5-minute run and validate the PAUSE behavioral response with the Threat firmware from Lab Handout 1-G.

## 2 Prerequisites

- Lab Handouts 1-A, 1-B, 1-C, 1-D complete.
- Target A and Target B from LH1-D running ping-pong successfully (≥95% ACK rate at 1 m).
- Lab Handout 1-G (interceptor / Threat firmware) is required for the PAUSE behavioral validation in Step 7. If LH1-G is not yet complete, finish LH1-G first; the two handouts are co-validated.

**LH1-F, LH1-G, and LH1-H are tightly interconnected — read this first**

The three Module 1 firmware/integration handouts cannot be *fully* completed in isolation. Plan to work through all three in alternation; expect to revisit each as the others come online.

- **LH1-F (this handout)** produces the cooperative-target firmware for two boards and the DATA/ACK ping-pong traffic source. The PAUSE listener at the end of LH1-F (Step 7, "I'm Jammed" behavior) cannot be validated until LH1-G is built and transmitting PAUSE packets at the Targets.
- **LH1-G (interceptor / Threat firmware)** consumes the LH1-F ping-pong traffic as its capture source. It cannot be validated until LH1-F Steps 1–6 (DATA/ACK without the PAUSE listener) are running. LH1-G's host-side parser step requires a USB-to-UART adapter; if one is unavailable, LH1-G is considered "code-complete" once it builds and the on-board log shows packets being received — the binary frame stream is then validated by LH1-H.
- **LH1-H (FPGA ingestion)** consumes the LH1-G binary forwarding frames and presents an end-to-end validation that does not require any extra host hardware (the Nexys A7 seven-segment display acts as a frame counter). It cannot be validated until LH1-G is producing frames on `USART3 TX`.

Recommended pass order: complete LH1-F Steps 1–6 (DATA/ACK only), then build LH1-G end-to-end against that traffic, then return here to finish LH1-F Step 7 (PAUSE listener), and finally close out with LH1-H. The Module 1 deliverable is the union of all three.

## 3 Packet Format Reference

Use the canonical 32-byte format from the Module 1 lesson plan (Table 4). Reproduced here verbatim — do not reorder or resize fields; downstream parsers in Module 2 and beyond index these offsets directly:

| Offset | Field | Length |
|---|---|---|
| 0–1 | Preamble sync (`0xAE5A`, big-endian) | 2 |
| 2 | Beacon ID (sender) | 1 |
| 3–4 | Sequence number (uint16, LE) | 2 |
| 5 | `TX` power (dBm, uint8) | 1 |
| 6 | `Modulation Code`: bits[7:2] = mod kind $\ll$ 2 (`0x01`=LoRa, `0x02`=Frequency Shift Keying (FSK), `0x03`=On-Off Keying (OOK)); bits[1:0] = pkt type (`00`=DATA, `01`=ACK, `10`=PAUSE, `11`=reserved) | 1 |
| 7 | Spreading factor (LoRa SF 7–12; `0x00` if non-LoRa) | 1 |
| 8 | Bandwidth code (`0x00`=125 kHz, `0x01`=250 kHz, ...) | 1 |
| 9–12 | Timestamp (uint32, LE, ms since boot) | 4 |
| 13–29 | Payload (DATA/ACK = `0xAA` fill) or PAUSE args (PAUSE: byte 13 = target ID, byte 14 = hold-off in 100 ms units, bytes 15–29 = `0xAA`) | 17 |
| 30–31 | Cyclic Redundancy Check (CRC)-16/CCITT-FALSE over bytes 0–29 (MSB-first) | 2 |

# 4 FSM Specification

Both Target A and Target B run the same shared FSM skeleton. The role-specific differences are confined to the `IDLE`, `BUILD`, and `WAIT_ACK` state implementations; everything else is identical:

1. **INIT**: Hardware Abstraction Layer (HAL) initialization, clock setup, peripheral configuration.
2. **CONFIG**: reset SX1276, verify `RegVersion`, program all LoRa registers per Table 1, put modem in `STANDBY`, install the `DIO0 EXTI` ISR.
3. **IDLE**: Target A: poll `ms_count` against `next_tx_ms`; transition to `BUILD_DATA` when due. Target B: enter `RXCONTINUOUS`; remain in `IDLE` until the ISR sets a `rx_ready` flag; transition to either `BUILD_ACK` (if the received packet is a DATA from a peer Beacon ID) or back to `IDLE` (if the packet is anything else — PAUSE addressed to a different ID, etc.). Both: at every `IDLE` entry, check `paused_until_ms`; if past expiry, clear the paused flag.
4. **BUILD_DATA** (Target A): populate the 32-byte packet with `pkt_type=DATA`, `mod_kind=LoRa`, current `seq`, and a structured payload (see Step 2 below). Compute CRC-16. Increment `seq`.
5. **BUILD_ACK** (Target B): populate the 32-byte packet with `pkt_type=ACK`, `mod_kind=LoRa`, the received DATA's `seq` echoed into the sequence field, and `0xAA` payload. Compute CRC-16.
6. **TX**: write `FIFO`, command `TX` mode, wait for `TxDone` with a 500 ms timeout. On timeout, transition to `FAULT`.
7. **WAIT_ACK** (Target A only): switch radio into `RXCONTINUOUS`; wait up to 200 ms for an ACK from Target B with matching `seq`. On match, capture the round-trip time. On timeout, log a missed-ACK event and continue.
8. **LOG**: `printf` a single-line status (CSV format consistent with LH1-D) over USART2 and return to `IDLE`.
9. **PAUSED** (entered by the `DIO0` ISR or main-loop poll on receipt of a PAUSE addressed to this target's ID): set `paused_until_ms = HAL_GetTick() + (pause_payload[1] × 100)`. Print `"I'm Jammed (by ID=0xFF, until t=%lu)"` on USART2 *only on the first entry per pause window* (throttled, not per-tick). On subsequent ISR PAUSE refreshes within the window, only update `paused_until_ms`; do not reprint. Skip all DATA TX (Target A) while paused. ACK TX (Target B) is permitted because it is a response to a real DATA reception. Return to `IDLE` on next main-loop iteration.
10. **FAULT**: `printf` an error line over USART2; clear all radio IRQ flags; force `CONFIG` state.

Table 1: LoRa parameters (same as LH1-D for compatibility).

| **Parameter** | **Value** |
|---|---|
| Carrier frequency | 906.5 MHz |
| Spreading factor | SF7 (Target A and Target B share SF) |
| Bandwidth | 125 kHz |
| Coding rate | 4/5 |
| CRC | On |
| Sync word | `0x34` |
| `TX` power | +14 dBm |

## 5 Shared Definitions (paste into both projects)

The following constants and helper prototypes are referenced throughout the FSM code in Steps 3–5. Add them once to a new header file `Core/Inc/lh1f_defs.h` in *both* the `TargetA` and `TargetB` projects, then `#include "lh1f_defs.h"` from `main.c`.

```c
#ifndef LH1F_DEFS_H
#define LH1F_DEFS_H

#include <stdint.h>

/* --- Beacon IDs (same in all three firmware projects) --- */
#define BEACON_ID_A        0x01u
#define BEACON_ID_B        0x02u
#define BEACON_ID_THREAT   0xFFu

/* --- Packet types (low 2 bits of Modulation Code byte) --- */
#define PKT_TYPE_DATA      0x00u
#define PKT_TYPE_ACK       0x01u
#define PKT_TYPE_PAUSE     0x02u

/* --- Modulation kinds (high 6 bits of Modulation Code byte, pre-shift) --- */
#define MOD_KIND_LORA      0x01u
#define MOD_KIND_FSK       0x02u
#define MOD_KIND_OOK       0x03u

/* --- LoRa carrier register triple for 906.5 MHz on 32 MHz TCXO --- */
#define RFM_FRF_MSB        0xE2u   /* RegFrfMsb  (0x06) */
#define RFM_FRF_MID        0xA0u   /* RegFrfMid  (0x07) */
#define RFM_FRF_LSB        0x00u   /* RegFrfLsb  (0x08) */

/* --- Helpers from LH1-D (already in your project; do not redefine) --- */
uint8_t  rfm_read_reg(uint8_t addr);
void     rfm_write_reg(uint8_t addr, uint8_t value);
void     rfm_set_mode_standby(void);
void     rfm_set_mode_rx(void);
void     lora_init(void);

/* --- New helpers introduced by LH1-F --- */
uint16_t crc16_ccitt(const uint8_t *buf, uint32_t len);
void     build_packet(uint8_t out[32], uint8_t my_id, uint16_t seq,
                      uint8_t pkt_type, uint8_t mod_kind);

#endif /* LH1F_DEFS_H */
```

Listing 1: `Core/Inc/lh1f_defs.h` — shared constants and prototypes (identical in both projects).

**Delete the LH1-D copies of `crc16_ccitt` and `build_packet` first**

LH1-D placed inline definitions of `crc16_ccitt` and `build_packet` (and their forward declarations) directly in `Core/Src/main.c`. LH1-F moves both into the dedicated `lh1f_packet.c` file shown below. Before adding the new file, open `Core/Src/main.c` in *both* the Target A and Target B projects and delete:

- the two forward declarations (`uint16_t crc16_ccitt(...);` and `void build_packet(...);`) inside the `/* USER CODE BEGIN PFP */` block,
- the full `crc16_ccitt` function body inside `/* USER CODE BEGIN 4 */`,
- the full `build_packet` function body inside `/* USER CODE BEGIN 4 */`.

The declarations now come from `lh1f_defs.h` (above) and the definitions from `lh1f_packet.c` (below). Skipping this step produces a multiple-definition linker error.

```c
#include "lh1f_defs.h"
#include "main.h"

extern volatile uint32_t ms_count;

/* CRC-16/CCITT-FALSE: poly=0x1021, init=0xFFFF, no reflect, xorout=0x0000.
 * Verified against the standard test vector: crc16_ccitt("123456789", 9)
 * must equal 0x29B1.
 */
uint16_t crc16_ccitt(const uint8_t *buf, uint32_t len) {
    uint16_t crc = 0xFFFFu;
    for (uint32_t i = 0; i < len; i++) {
        crc ^= ((uint16_t)buf[i]) << 8;
        for (int b = 0; b < 8; b++) {
            crc = (crc & 0x8000u) ? (uint16_t)((crc << 1) ^ 0x1021u)
                                  : (uint16_t)(crc << 1);
        }
    }
    return crc;
}

/* build_packet: populate the canonical 32-byte radio packet.
 * Caller is responsible for any payload customization AFTER this call
 * (e.g. PAUSE adds target-id/hold-off in bytes 13-14 then RECOMPUTES CRC).
 */
void build_packet(uint8_t out[32], uint8_t my_id, uint16_t seq,
                  uint8_t pkt_type, uint8_t mod_kind)
{
    /* Preamble */
    out[0] = 0xAEu;
    out[1] = 0x5Au;
    /* Sender + sequence */
    out[2] = my_id;
    out[3] = (uint8_t)(seq & 0xFFu);
    out[4] = (uint8_t)((seq >> 8) & 0xFFu);
    /* TX power */
    out[5] = 14u;
    /* Packed Modulation Code byte: bits[7:2] = mod_kind << 2,
     *                              bits[1:0] = pkt_type             */
    out[6] = (uint8_t)((mod_kind << 2) | (pkt_type & 0x03u));
    /* Spreading factor / bandwidth code */
    out[7] = (mod_kind == MOD_KIND_LORA) ? 7u : 0u;   /* SF7 default */
    out[8] = 0u;                                       /* 125 kHz BW  */
    /* Timestamp */
    uint32_t t = ms_count;
    out[9]  = (uint8_t)(t      );
    out[10] = (uint8_t)(t >>  8);
    out[11] = (uint8_t)(t >> 16);
    out[12] = (uint8_t)(t >> 24);
    /* Default payload: 0xAA filler. Caller may overwrite bytes 13-29
     * before transmit (e.g. for a PAUSE packet's target-id field).    */
    for (int i = 13; i < 30; i++) out[i] = 0xAAu;
    /* CRC-16/CCITT-FALSE over bytes 0-29, MSB first */
    uint16_t crc = crc16_ccitt(out, 30);
    out[30] = (uint8_t)(crc >> 8);
    out[31] = (uint8_t)(crc);
}
```

Listing 2: `Core/Src/lh1f_packet.c` — create this new file in both projects. Implements the CRC and the canonical packet builder used by every `BUILD_*` state.

**CRC unit-test deliverable**

The handout deliverable requires a unit test of `crc16_ccitt` against a known vector. Add the following one-shot self-check inside `/* USER CODE BEGIN 2 */` (run once at boot, before the main loop):

```
{
    const uint8_t vec[9] = {'1','2','3','4','5','6','7','8','9'};
    uint16_t got = crc16_ccitt(vec, 9);
    if (got != 0x29B1u) {
        printf("CRC16 SELFTEST FAIL: expected 0x29B1 got 0x%04X\r\n", got);
        while (1) { /* hang on failure */ }
    } else {
        printf("CRC16 SELFTEST PASS (0x29B1)\r\n");
    }
}
```

### `printf` retarget (one-time setup, both projects)

The FSM relies on `printf` reaching USART2. If you have not already done so during LH1-D, add the following retarget shim to `main.c` in both projects (outside any `USER CODE` block, near the top after the includes; the `HAL_UART_Transmit` reference uses `huart2` declared by CubeMX):

```
#include <stdio.h>
extern UART_HandleTypeDef huart2;

int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
/* Newlib-nano hook: some toolchains route stdout through _write instead. */
int _write(int file, char *ptr, int len) {
    (void)file;
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);
    return len;
}
```

Listing 3: `printf` retarget to USART2. Required by every `printf` call in the FSM.

In the linker settings (CMakePresets or CubeMX project-level), confirm `-u _printf_float` is *not* enabled (we do not use `%f`) and that `Use float with printf from newlib-nano` is unchecked. This keeps the binary small and the per-call cost of `printf` bounded.

# 6 Procedure

## 6.1 Step 1 — Open the existing TargetA and TargetB projects

The LH1-D `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` projects continue to be the source-of-truth for the cooperative-target firmware. **Do not copy, rename, or move them, and do not run CubeMX again.** The LH1-F additions extend the LH1-D code with a proper FSM, the PAUSE listener path, and the DIO0 `EXTI` interrupt handler — all of these are pure C-level edits inside the existing USER CODE blocks. The CubeMX peripheral configuration was already done in LH1-C Step 1 (SPI1, USART2, USART3 with DMA, TIM2 1 ms tick, `RFM_CS`/`RFM_RST`/`RFM_DIO0` GPIO labels with EXTI on PA8 and the EXTI[9:5] NVIC line enabled). No re-generation is required.

1. Open both `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` in separate VS Code windows (same setup as LH1-D).
2. Press `F7` in each window to confirm both projects still build with the LH1-D code in place.

## 6.2 Step 2 — Wire the TIM2 1 ms tick into the C code

CubeMX (LH1-C Step 1.7) already enabled `TIM2` as a 1 ms periodic timer and enabled its NVIC interrupt; the C-side `ms_count` variable and the `HAL_TIM_PeriodElapsedCallback` override still need to be added.

1. In both `TargetA` and `TargetB` `main.c` files, add to `/* USER CODE BEGIN PV */`:

```
/* USER CODE BEGIN PV */
volatile uint32_t ms_count = 0;
/* USER CODE END PV */
```

2. Add the callback in `/* USER CODE BEGIN 4 */`:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) ms_count++;
}
```

3. Inside `/* USER CODE BEGIN 2 */` (after `lora_init()`), start the timer in interrupt mode:

```
HAL_TIM_Base_Start_IT(&htim2);
```

4. In Target A only, schedule the DATA cadence: store `static uint32_t next_tx_ms` in the main loop and advance it by 1000 ms after each successful `BUILD_DATA`→`TX`→`WAIT_ACK`→`LOG` cycle. Target B does not schedule transmissions; it only responds.

### 6.3 Step 3 — DIO0 EXTI ISR and shared receive path

Both targets handle the SX1276's `RxDone` interrupt via the `DIO0` pin's `EXTI` line. Implement the ISR in `/* USER CODE BEGIN 4 */`:

```
typedef struct {
    uint8_t  raw[32];
    uint32_t rx_ts_ms;
    int8_t   rssi_dbm;
    int8_t   snr_q025;
} pkt_rx_t;

volatile pkt_rx_t  rx_slot;
volatile uint8_t   rx_ready = 0;
volatile uint32_t  paused_until_ms = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin != RFM_DIO0_Pin) return;

    uint8_t irq = rfm_read_reg(0x12);
    if ((irq & 0x40) == 0) {
        rfm_write_reg(0x12, 0xFF);
        return;
    }
    if (irq & 0x20) {  /* CRC error */
        rfm_write_reg(0x12, 0xFF);
        return;
    }

    /* Read the packet into rx_slot */
    uint8_t rx_addr = rfm_read_reg(0x10);
    rfm_write_reg(0x0D, rx_addr);
    for (int i = 0; i < 32; i++) {
        ((uint8_t*)rx_slot.raw)[i] = rfm_read_reg(0x00);
    }
    rx_slot.rx_ts_ms = ms_count;
    rx_slot.rssi_dbm = (int8_t)((int16_t)rfm_read_reg(0x1A) - 157);
    rx_slot.snr_q025 = (int8_t)rfm_read_reg(0x19);
    rfm_write_reg(0x12, 0xFF);

    /* Check for PAUSE addressed to us --- handled in ISR for low latency */
    uint8_t sender   = rx_slot.raw[2];
    uint8_t pkt_type = rx_slot.raw[6] & 0x03;
    if (pkt_type == PKT_TYPE_PAUSE
        && sender == BEACON_ID_THREAT
        && rx_slot.raw[13] == MY_BEACON_ID) {
        uint8_t units = rx_slot.raw[14];
        paused_until_ms = ms_count + ((uint32_t)units * 100);
        /* Defer printf to main loop to avoid blocking ISR */
```

```
        rx_ready = 1;  /* main loop notices the PAUSE and prints once */
        return;
    }

    rx_ready = 1;  /* main loop will dispatch DATA / ACK */
}
```

The constant `MY_BEACON_ID` is defined per project: Target A defines `#define MY_BEACON_ID BEACON_ID_A`, Target B defines `#define MY_BEACON_ID BEACON_ID_B`.

### 6.4 Step 4 — Main-loop FSM skeleton

In `/* USER CODE BEGIN WHILE */`, replace the LH1-D inline-loop logic with the explicit FSM:

```
typedef enum {
    ST_IDLE, ST_BUILD_DATA, ST_BUILD_ACK, ST_TX,
    ST_WAIT_ACK, ST_LOG, ST_FAULT
} state_t;

state_t state = ST_IDLE;
uint16_t seq = 0;
uint16_t pending_ack_seq = 0;
uint32_t t_tx_ms = 0;
uint8_t  pkt[32];
uint8_t  log_was_acked = 0;
int16_t  log_ack_rssi = 0;
uint32_t log_rtt_ms = 0;

static uint8_t was_paused = 0;

while (1) {
    /* Honor PAUSE: if paused, suppress DATA scheduling but still
       process incoming RX (so ACKs can still be sent and PAUSE
       refreshes can extend the hold-off). */
    uint8_t paused_now = (ms_count < paused_until_ms);

    /* Throttled "I'm Jammed" print: emit only on the rising edge of
       paused_now (entry into a pause window). Subsequent PAUSE refreshes
       from the Threat extend paused_until_ms but do NOT re-print. The
       was_paused latch clears when the hold-off expires, so the next
       fresh pause window prints again. */
    if (paused_now && !was_paused) {
        printf("I'm Jammed (by ID=0x%02X, until t=%lu)\r\n",
               BEACON_ID_THREAT, (unsigned long)paused_until_ms);
    }
    was_paused = paused_now;

    switch (state) {
    case ST_IDLE:
```

```
        /* Target A initiator path */
#ifdef ROLE_INITIATOR
        if (!paused_now && ms_count >= next_tx_ms) {
            state = ST_BUILD_DATA;
            break;
        }
#endif
        /* Both: handle any received packet */
        if (rx_ready) {
            rx_ready = 0;
            uint8_t pkt_type = rx_slot.raw[6] & 0x03;
            uint8_t sender   = rx_slot.raw[2];

#ifdef ROLE_RESPONDER
            if (pkt_type == PKT_TYPE_DATA && sender == BEACON_ID_A) {
                pending_ack_seq = (uint16_t)rx_slot.raw[3]
                                | ((uint16_t)rx_slot.raw[4] << 8);
                state = ST_BUILD_ACK;
                break;
            }
#endif
            /* Target A waiting for an ACK while in IDLE means the
               WAIT_ACK timeout already expired; ignore stray ACKs. */
            (void)pkt_type; (void)sender;
        }
        break;

    case ST_BUILD_DATA:
        build_packet(pkt, MY_BEACON_ID, seq,
                     PKT_TYPE_DATA, MOD_KIND_LORA);
        state = ST_TX;
        break;

    case ST_BUILD_ACK:
        build_packet(pkt, MY_BEACON_ID, pending_ack_seq,
                     PKT_TYPE_ACK, MOD_KIND_LORA);
        state = ST_TX;
        break;

    case ST_TX: {
        rfm_set_mode_standby();
        rfm_write_reg(0x0D, rfm_read_reg(0x0E));
        for (int i = 0; i < 32; i++) rfm_write_reg(0x00, pkt[i]);
        rfm_write_reg(0x22, 32);
        t_tx_ms = ms_count;
        rfm_write_reg(0x12, 0xFF);
        rfm_write_reg(0x01, 0x83);  /* TX */
```

```
        uint32_t tx_deadline = ms_count + 500;
        uint8_t ok = 0;
        while (ms_count < tx_deadline) {
            if (rfm_read_reg(0x12) & 0x08) { ok = 1; break; }
        }
        rfm_write_reg(0x12, 0xFF);
        if (!ok) { state = ST_FAULT; break; }

#ifdef ROLE_INITIATOR
        rfm_set_mode_rx();
        state = ST_WAIT_ACK;
#else
        rfm_set_mode_rx();
        state = ST_LOG;
#endif
        break;
    }

#ifdef ROLE_INITIATOR
    case ST_WAIT_ACK: {
        uint32_t deadline = ms_count + 200;
        log_was_acked = 0;
        while (ms_count < deadline) {
            if (rx_ready) {
                rx_ready = 0;
                uint8_t pkt_type = rx_slot.raw[6] & 0x03;
                uint8_t sender   = rx_slot.raw[2];
                uint16_t rs = (uint16_t)rx_slot.raw[3]
                            | ((uint16_t)rx_slot.raw[4] << 8);
                if (pkt_type == PKT_TYPE_ACK
                    && sender == BEACON_ID_B
                    && rs == seq) {
                    log_was_acked = 1;
                    log_ack_rssi  = rx_slot.rssi_dbm;
                    log_rtt_ms    = ms_count - t_tx_ms;
                    break;
                }
            }
        }
        next_tx_ms = t_tx_ms + 1000;  /* schedule next on a fixed grid */
        seq++;
        state = ST_LOG;
        break;
    }
#endif

    case ST_LOG:
#ifdef ROLE_INITIATOR
```

```
        if (log_was_acked) {
            printf("CSV,%u,%lu,%lu,%lu,%d\r\n",
                   (unsigned)(seq - 1), (unsigned long)t_tx_ms,
                   (unsigned long)(t_tx_ms + log_rtt_ms),
                   (unsigned long)log_rtt_ms, (int)log_ack_rssi);
        } else {
            printf("CSV,%u,%lu,0,0,0  (no ACK)\r\n",
                   (unsigned)(seq - 1), (unsigned long)t_tx_ms);
        }
#else
        printf("ACK seq=%u rssi=%d\r\n",
               (unsigned)pending_ack_seq, (int)rx_slot.rssi_dbm);
#endif
        state = ST_IDLE;
        break;

    case ST_FAULT:
        printf("FAULT --- TX timeout, reconfiguring radio\r\n");
        rfm_write_reg(0x12, 0xFF);
        lora_init();
        rfm_set_mode_rx();
        state = ST_IDLE;
        break;
    }
}
```

### 6.5 Step 5 — Define the role and Beacon ID per project

The same FSM source above compiles into either Target A or Target B firmware via two compile-time defines per project. Because LH1-C built both projects with the VS Code STM32 extension's CMake flow, set the defines in `CMakeLists.txt` (do not re-run CubeMX). Open each project's `CMakeLists.txt`, find the existing `target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE ...)` call (CubeMX generates one with `USE_HAL_DRIVER` and `STM32L476xx`) and append the two role lines.

**Target A project — in** `Nucleo_RFM95W_TargetA/CMakeLists.txt`:

```
target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE
    USE_HAL_DRIVER
    STM32L476xx
    ROLE_INITIATOR
    MY_BEACON_ID=BEACON_ID_A
)
```

Listing 4: Append to the existing `target_compile_definitions` call.

**Target B project — in** `Nucleo_RFM95W_TargetB/CMakeLists.txt`:

```
target_compile_definitions(${CMAKE_PROJECT_NAME} PRIVATE
    USE_HAL_DRIVER
    STM32L476xx
    ROLE_RESPONDER
    MY_BEACON_ID=BEACON_ID_B
)
```

After editing `CMakeLists.txt`, run **Clean Reconfigure** from the VS Code STM32 extension's Build menu (or delete the `build/` directory) so CMake re-discovers the new defines. The `#ifdef ROLE_INITIATOR` / `#ifdef ROLE_RESPONDER` guards in the FSM above then compile each project into its role.

### 6.6 Step 6 — GPIO instrumentation pulse

Add a GPIO output pulse on the entry to the `TX` state. This gives a scope trigger for measuring transmission timing in Module 3.

To avoid re-running CubeMX (which can break the project's CMake state), configure the trigger pin entirely from C using direct HAL GPIO calls. `PB10` is free on the Nucleo-L476RG morpho header (CN10 pin 25) and is not used by any peripheral configured in LH1-C.

1. In `/* USER CODE BEGIN 2 */`, before the main loop, configure `PB10` as a push-pull output:

   ```
   __HAL_RCC_GPIOB_CLK_ENABLE();
   GPIO_InitTypeDef trig = {0};
   trig.Pin   = GPIO_PIN_10;
   trig.Mode  = GPIO_MODE_OUTPUT_PP;
   trig.Pull  = GPIO_NOPULL;
   trig.Speed = GPIO_SPEED_FREQ_HIGH;
   HAL_GPIO_Init(GPIOB, &trig);
   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);
   ```

2. In the `ST_TX` case, immediately before commanding `TX` mode, raise the pin:

   ```
   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_SET);
   ```

   After the `TxDone` flag clears (i.e., at the bottom of the `ST_TX` case before transitioning out), lower it:

   ```
   HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);
   ```

3. Wire a scope probe to Nucleo CN10 pin 25 (PB10), with ground on any GND pin (e.g., CN10 pin 9). A scope or logic analyzer on this pin should show evenly-spaced pulses (1 s period for Target A) when the firmware is not paused, and silence on Target A while the Threat's user button is held.

### 6.7 Step 7 — Co-validate with the Threat from Lab Handout 1-G

1. Flash Target A and Target B from this handout. Flash the Threat firmware from Lab Handout 1-G.
2. Confirm the ping-pong baseline: 100% reception on Target B over a 5-minute run, no CRC errors, contiguous sequence numbers, no `FAULT` entries.
3. With the ping-pong running, hold the user button (B1, blue button) on the Threat board for 3 s. Verify on Target A's USART2:
   - `"I'm Jammed (by ID=0xFF, until t=...)"` appears within ∼100 ms of button press.
   - `CSV,...` log lines from Target A stop appearing.
   - Within ∼500 ms of button release, `CSV,...` log lines resume.
4. Capture the Target A USART2 log across the full sequence and save as `LH1F_pause_response.log`.

**[Figure placeholder]**

`LH1F_pause_response_log.png`

*Annotated Target A USART2 capture during a 3-s Threat button-hold: per-second `CSV` lines, then a single `"I'm Jammed"` line, then silence for 3 s, then `CSV` lines resuming.*

Figure 1: Annotated Target A USART2 capture during a 3-s Threat button-hold: per-second `CSV` lines, then a single `"I'm Jammed"` line, then silence for 3 s, then `CSV` lines resuming.

## 7 Verification Checklist

- ☐ Both projects build with zero errors after the `DIO0 EXTI` reconfiguration.
- ☐ 5-minute ping-pong run: 100% ACK reception on Target A across 300 transmissions, no CRC errors, contiguous sequence numbers, no `FAULT` entries.
- ☐ Scope pulse on the instrumentation GPIO shows stable 1 s period (Target A) within ±5 ms.
- ☐ PAUSE behavioral response: `"I'm Jammed"` appears within ∼100 ms of Threat button press; DATA TX stops; resumes within ∼500 ms of button release.
- ☐ Target B's USART2 log shows no `"I'm Jammed"` entries during the Step 7 test (assuming Target A is the higher-RSSI station from the Threat's perspective).

**Lab Handout 1-F Deliverable**

1. Full source code for both `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` projects, with the shared FSM clearly structured and the `#ifdef ROLE_*` guards visible.
2. CRC-16 implementation (standalone function) with a unit test against a known vector.
3. 5-minute ping-pong CSV log from Target A: ≥300 rows, ACK rate, RTT statistics, sequence-gap count.
4. PAUSE behavioral-response log (`LH1F_pause_response.log`) showing the full `CSV` → `"I'm Jammed"` → silence → `CSV` resume sequence.
5. Oscilloscope screenshot of the GPIO instrumentation pulse showing stable period for at least 5 transmissions, plus a screenshot of the same line going silent during a Threat button-hold and resuming after release.

# Glossary

| | | |
|---|---|---|
| **CRC** | Cyclic Redundancy Check | 3, 4, 5, 17, 18 |
| **FSK** | Frequency Shift Keying | 3 |
| **FSM** | Finite State Machine | 1, 4, 6, 9, 10, 12, 16, 18 |
| **HAL** | Hardware Abstraction Layer | 4, 17 |
| **OOK** | On-Off Keying | 3 |
| **RF** | Radio Frequency | 1 |